



Pooz

Technical Documentation — Decentralized Prediction Markets on VeChain

Pooz (code name **PredictChain**) is a non-custodial, on-chain prediction market platform where users stake VET on the outcome of real-world events. Every stake is a verifiable VeChain transaction; every payout is executed by smart-contract logic; every position is transparent on-chain.

VeChain Network

Connex SDK

VeWorld Wallet

React 18

Socket.IO Realtime

Non-Custodial

Document	System & Architecture Reference
Frontend repo	predictchain-frontend
Scope	Architecture · On-chain integration · Market engine · Wallet & Auth · Real-time · Admin · Developer guide
Status	Working draft — generated from current source tree

Contents

1 Executive Summary

2 High-Level Architecture

3 On-Chain System & Smart-Contract Integration

3.1 Contract Surface & Function Selectors

3.2 Transaction Encoding & Receipt Flow

3.3 On-Chain Lifecycle of a Market

4 Prediction Market Engine

4.1 Market Model & Categories

4.2 Odds & Payout Mechanics

4.3 Staking, Claims & Refunds

4.4 Fee Model

5 Web3 Connectivity & VeWorld Wallet Integration

6 Authentication & Session System

7 Real-Time Data Pipeline

8 Admin Panel & Governance

9 Frontend Architecture

9.1 Routing & Page Map

9.2 Global State & Data Hooks

9.3 Landing Experience

9.4 Dashboard Experience

10 Developer Guide & Local Setup

11 Appendix

1 Executive Summary

Pooz is an on-chain prediction market built on the **VeChain** blockchain. Users stake **VET** on the YES or NO outcome of real-world event questions across Crypto, Sports, Politics, Entertainment, Gaming and Stocks. The platform couples a fast, modern browser frontend with the VeChain **Connex** SDK and the **VeWorld** wallet to offer a completely **non-custodial** experience: player principal never leaves their own wallet until a payout is sent directly by the smart contract.

Unlike many centralized "sportsbook" UIs that wrap an off-chain ledger, Pooz treats every prediction lifecycle step as a first-class on-chain action:

- **Market registration** — an admin signs a `createMarket` transaction that persists the question, category, close time and featured flag on-chain; the returned contract market id is stored in the backend DB.
- **Staking** — a user signs a `predict` transaction that transfers stake VET (as wei `value`) to the prediction contract and records a YES/NO position.
- **Resolution** — an admin signs a `resolveMarket` transaction which finalizes the winning side and enables payouts.
- **Claims & refunds** — winners call `claim` to receive proportional payouts; cancelled-market participants call `refund`.
- **Fee withdrawal** — the accumulated 1.5% platform fee is withdrawn on-chain by the admin to a VeChain address.

Because gas on VeChain costs fractions of a cent (Proof of Authority, ~10-second finality), even small precision bets are economically viable — a central differentiator the product leans on (micro-transaction ready, near-instant settlement).

1.1 Product Positioning

Non-custodial

Funds remain in the user's wallet; the contract escrows only pooled stake for the market's life.

On-chain clarity

Every bet, outcome and payout is verifiable by anyone on the VeChain explorer.

Instant settlements

Correct calls are paid automatically through `claim()` within seconds of resolution.

Audited, open contracts

Product marketing states independently audited, open-source smart contracts.

Micro-transaction ready

Fractional-cent gas enables stakes of 1+ VET without fee friction.

Enterprise-grade network

VeChain Proof-of-Authority infrastructure — low energy, trusted by enterprise supply chains.

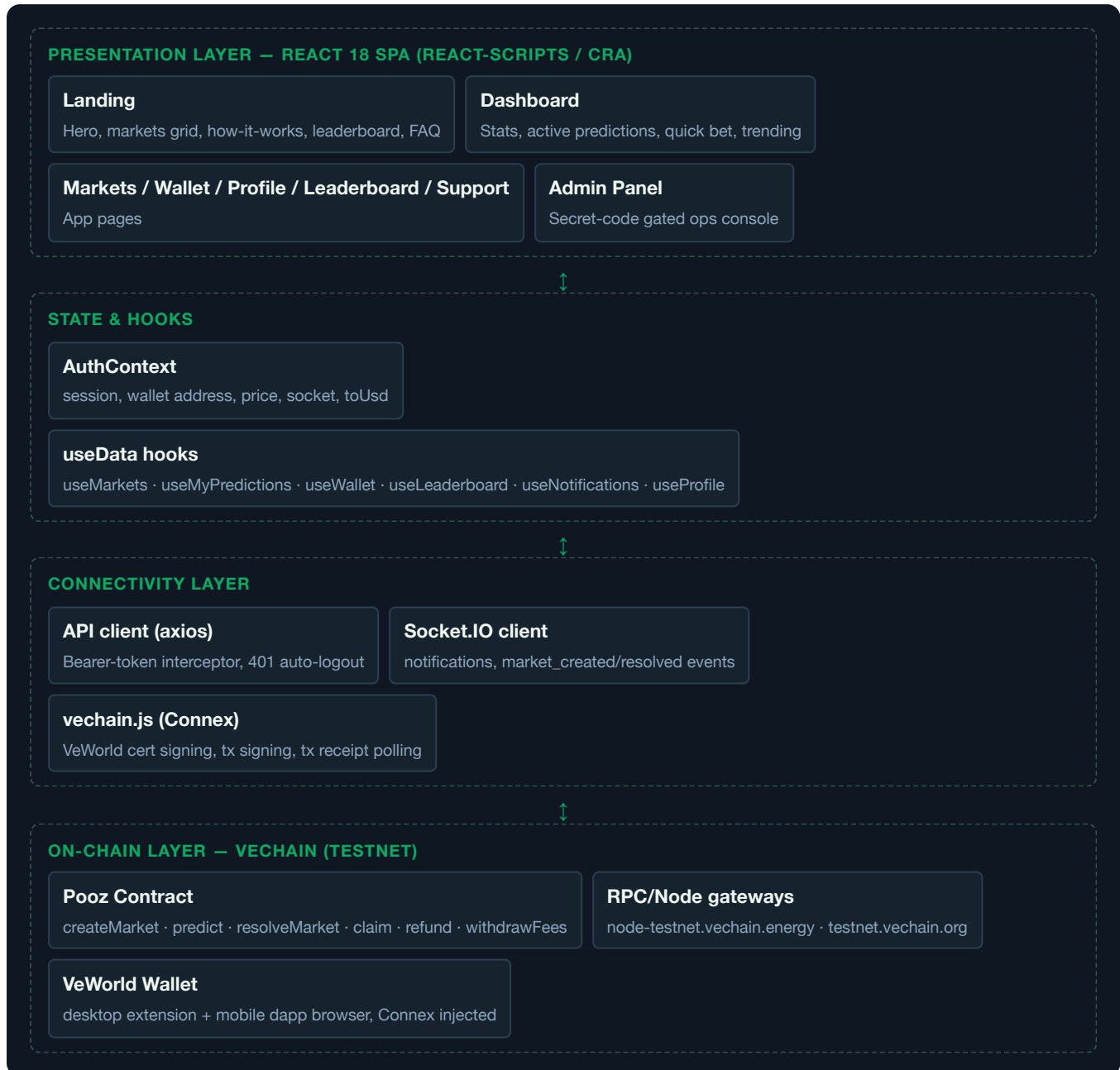
1.2 Headline Numbers (public-facing marketing)

METRIC	VALUE	WHERE SURFACED
Live markets	240+	Landing hero, stats cube
Active predictors	10,000+	Landing proof strip
VET settled on-chain	2M+ VET	Landing stats cube
Market resolution rate	99.8%	Landing globe / cube
Platform fee	1.5% on resolved pools	Predict modal, admin panel

Branding note. This document brands the product as **POOZ**. The current codebase, API namespace and session copies still surface legacy identifiers — **PredictChain** (e.g. loading screen "Loading PredictChain...", shared tweet text "Predict on PredictChain") and **VETPREDICT** (landing page header, favicon/theme meta) — and should be migrated to the POOZ brand across the landing page, copy, repo name and API messages.

2 High-Level Architecture

The platform is organized in four synchronized tiers: a **presentation layer** (React SPA), a **connectivity layer** (REST API client, Socket.IO, Connex/VeWorld), an **on-chain layer** (VeChain testnet/node RPC + the prediction smart contract), and an **off-chain backend** (REST + realtime) that mirrors on-chain events into relational state the UI can read cheaply.



2.1 Framework & Stack Selection

CONCERN	TECHNOLOGY	RATIONALE / ROLE
UI framework	React 18.2.0, react-dom	Component model, concurrent-safe data fetching via hooks.
Build tooling	react-scripts 5.0.1 (CRA)	Zero-config dev server & production build; <code>npm start</code> / <code>npm run build</code> .

Styling	Tailwind CSS 4.2.2, CSS-in-JS	Tailwind available; most UI ships as injected <code><style></code> blobs (Dashboard/Admin) with CSS custom properties for theming.
Blockchain	@vechain/connex 2.3.2	Low-level client/signer surface for VeChain; exposed by VeWorld injection as <code>window.connex</code> / <code>window.vechain</code> .
HTTP	axios 1.6.2	REST client with interceptors for JWT auth and 401 handling.
Realtime	socket.io-client 4.6.2	Server-pushed notifications and market-update broadcast.
Icons	lucide-react	Icon set.

Backend contract. The backend (Express REST + Socket.IO + Mongo-style persistence) is reached at `REACT_APP_API_URL` (default `http://localhost:5001/api`) and is expected alongside, but not included in, this frontend tree. All endpoints referenced in this document are those the frontend depends on (see §11 API surface).

3 On-Chain System & Smart-Contract Integration

One audited prediction contract (configured by `REACT_APP_CONTRACT_ADDRESS`) exposes the entire game loop. The frontend never re-implements logic — it encodes function calls into VeChain clauses, has them signed by VeWorld, and mirrors confirmations into the backend DB.

3.1 Contract Surface & Function Selectors

The client invokes contract functions using their 4-byte selectors (`keccak256(signature)[0:4]`). The signatures are defined in `src/services/vechain.js` and `src/pages/admin/AdminPanel.js`:

FUNCTION	SELECTOR	PURPOSE	WHERE CALLED
<code>predict(uint256,bool)</code>	<code>0xebd389fe</code>	Record a stake for market id (YES=1/NO=0), with stake VET as <code>tx value</code> .	<code>vechain.js :: placePredictionOnChain</code>
<code>claim(uint256)</code>	<code>0x677bd9ff</code>	Withdraw proportional winnings for a won position.	<code>vechain.js :: claimWinningsOnChain</code>
<code>refund(uint256)</code>	<code>0x5b7baf64</code>	Return stake when a market is cancelled.	<code>vechain.js :: claimRefundOnChain</code>
<code>withdrawFees(address)</code>	<code>0x164e68de</code>	Send accumulated platform fees to an admin address.	<code>vechain.js :: withdrawFeesOnChain</code>
<code>createMarket(string,string,uint256,bool)</code>	<code>0xc48d8656</code>	Register a market on-chain (title, category, closesAt, featured); emits a topic the client parses as the market id.	<code>AdminPanel :: buildCreateMarketCalldata</code>
<code>resolveMarket(uint256,bool)</code>	<code>0x57bde446</code>	Finalize a market as YES or NO; unlocks payouts.	<code>AdminPanel :: SEL_RESOLVE_MARKET</code>

3.2 Transaction Encoding & Receipt Flow

Two encoding styles are used, reflecting the two generations of the integration:

- **Compact ABI (`predict` / `claim` / `refund` / `withdrawFees`).** Arguments are packed as 32-byte ABI slots concatenated after the selector into the clause `data`. For tolled calls the contract also carries a `value` clause equal to the stake in wei.
- **Offset-style ABI (`createMarket`).** Stricter Solidity `abi.encode` layout using base offsets (e.g. `0x80` header, string offsets resolved after the fixed tuple, dynamic-length-prefixed string payloads).

```

// stake path
data = "0x" + PREDICT_SIG
      + encodeUint256(contractMarketId)
offset
      + encodeBool(isYes)
title ptr
value = toHex(BigInt(stakeVet * 1e18))

// market creation path (calldata layout)
sel  = SEL_CREATE_MARKET
      + encodeUint256(128) // base
      + encodeUint256(128 + 32 + titleSlots*32) //
      + encodeUint256(closesAtUnix)
      + encodeBool(featured)
      + encodeStr(title) + encodeStr(category)

```

Every signed transaction is confirmed through receipt polling rather than fire-and-forget — see `waitForTx` (60 s budget, 4 s interval) which treats a `reverted: true` receipt as a hard failure, and the admin-path `waitForReceipt` variant (20×3 s).

3.3 On-Chain Lifecycle of a Market

STAGE	FRONTEND ACTION	ON-CHAIN EFFECT	OUTCOME STORED IN DB
Register	Admin creates market (form → calldata → Connex <code>sign("tx")</code>)	<code>createMarket(...)</code>	<code>contractMarketId</code> (parsed from receipt event topic), <code>txHash</code>
Resync / Push	Admin re-encodes an existing DB market	<code>createMarket(...)</code> again	<code>contractMarketId</code> / <code>txHash</code> refreshed
Stake	User confirms amount + YES/NO in VeWorld	<code>predict(id) + value=VET</code>	prediction with <code>txHash</code> , <code>side</code> , <code>stakeVet</code> , <code>contractPredictionId</code>
Resolve	Admin clicks Resolve as YES / NO	<code>resolveMarket(id, winner)</code>	<code>market.result</code> , status resolved, payouts computed
Claim	User clicks Claim on a won position	<code>claim(predictionId)</code>	<code>claimTxHash</code> ; payout flagged settled
Cancel	Admin cancels a market	<code>refund(id)</code> per participant	status cancelled, stakes refundable

VTHO gas check. VeChain transactions are paid in VTHO (energy). The claim path actively inspects failure messages for "VTHO"/"gas"/"fee" and surfaces a friendly "Get VTHO at faucet.vecha.in" hint instead of a raw error.

4 Prediction Market Engine

4.1 Market Model & Categories

A market is a binary (YES/NO) question with a close timestamp. The UI supports nine categories: **Sports**, **Crypto**, **Politics**, **Entertainment**, **Gaming**, **Stocks**, **Meme**, **Other** and an `All` view. Each market carries:

FIELD	MEANING
<code>title</code>	The binary question, e.g. "Will Arsenal win their next Premier League match?"
<code>category</code>	Filter bucket + colored badge mapping.
<code>closesAt</code>	UTC close; the dashboard renders fuzzy "x h / d left" and 404-guards expired pushes.
<code>yesPool / noPool / totalPool</code>	Cumulative VET staked per side; drives live odds bar.
<code>featured</code>	Homepage highlight (green border + "FEATURED" chip).
<code>contractMarketId</code>	On-chain market index emitted by <code>createMarket</code> ; required before users can stake.
<code>oracleSource</code>	Resolution provenance — manual (admin) today, Witnet oracle selectable.
<code>emoji</code>	Visual icon on cards and modals.

4.2 Odds & Payout Mechanics

Odds are computed dynamically from pooled liquidity (parimutuel-style):

```
YES%      = round(yesPool / totalPool * 100)  (fallback 50 when totalPool == 0)
YES odds  = 100 / YES%                        // e.g. 68% YES → 1.47x
NO odds   = 100 / (100 - YES%) // e.g. 32% NO  → 3.13x
Potential = stake * odds
```

Because odds move with every stake, the UI recomputes the modal summary live from the latest pool snapshot: stake, current odds, platform fee, and potential win (presented both in VET and USD via the live VET price).

4.3 Staking, Claims & Refunds

Staking flow

1. User chooses YES/NO, stakes ≥ 1 VET, and must hold a linked VeWorld wallet.
2. `placePredictionOnChain` builds a `predict(id, side)` clause with `value = stake * 1018`. A VeWorld-context check (`hasConnex`) and a `waitForConnex(5000)` guard precede signing.
3. After the "Confirm in VeWorld..." / "Waiting for confirmation..." states, `waitForTx` verifies the receipt; only then the prediction is recorded via the backend (`POST /predictions`), keeping DB and chain consistent.

Claim flow

1. The won prediction carries a `contractPredictionId` (also fallback `predictionIndex`).
2. `claimWinningsOnChain` signs `claim(id)` (zero value), polls the receipt, then `POST /predictions/:id/claim` persists the claim tx; the UI announces the payout in VET.

Refund flow

Cancelled markets make positions refundable via `refund(id)`, surfaced through the transaction history as a `refund` type.

4.4 Fee Model

The platform charges a single **1.5% fee** deducted from resolved pools at settlement ("1.5% of resolved pools", "Platform Fees (1.5%)"). Winners receive proportional payouts; the residual is aggregated in the contract and withdrawn by the admin in one on-chain `withdrawFees(adminWallet)` call. The admin overview reconciles fee figures between the backend stats and the on-chain `contract-stats` endpoint, taking the maximum of the two.

FLOW	EVENTUAL VALUE	NOTES
Stake	100% enters pool	No fee on the way in (non-custodial).
Win	$0.985 \times$ proportional payout	1.5% platform fee on resolved pools.
Cancel	100% refunded	Refund path, no fee.

Copy discrepancy. The landing FAQ copy mentions a "flat 2% platform fee on winnings" while the implemented modal and admin UI consistently use **1.5%**. The implementation (1.5%) is authoritative; the FAQ copy should be reconciled.

5 Web3 Connectivity & VeWorld Wallet Integration

5.1 Injection Model

VeChain wallets (primarily **VeWorld**) expose Connex through two shapes which the client normalizes (src/services/vechain.js):

ENVIRONMENT	INJECTION	SIGNING SURFACE USED
Desktop extension	window.connex	connex.vendor.sign("cert" "tx").request()
Mobile dapp browser	window.vechain	newConnexSigner(genesisId) → signCert / signTx
Mobile, no wallet	—	Redirect to veworld://browser?url=... deep link, fallback to veworld.net.

5.2 Network Fingerprint

```
GENESIS_ID (testnet) = 0x000000000b2bce3c70bc649a02749e8687721b09ed2e15997f466536b20bb127
NODE_URL (defaults) = https://node-testnet.vechain.energy // src/services/vechain.js
                    https://testnet.vechain.org // AdminPanel fallback
CONTRACT = REACT_APP_CONTRACT_ADDRESS // set via .env
```

5.3 Signing Primitives

- **Certificates** — identity/agreement proofs used for auth (getWalletAddress uses a signCert({purpose:"identification"}) and returns annex.signer; message auth signs a server-supplied nonce message with purpose:"agreement").
- **Transactions** — single-clause signed txs for predict/claim/refund/withdraw; multi-arg ABI payloads prepared client-side.

5.4 Non-Custodial Guarantees

The wallet is the only keeper of assets and keys. Contract calls move VET only via explicit user approvals inside VeWorld; the platform records transaction hashes but never holds funds, private keys or signing material. This property is the product's core trust pillar ("My VET never leaves my wallet until a payout hits").

5.5 Journey Guardrails

- waitForConnex(5000) — tolerates slow injection and wallet warm-up.
- isMobileBrowser() — if a mobile user has no wallet, auto-redirect into VeWorld and enumerate the app stores otherwise.
- ConnectWalletPrompt — guides email/Google users to link a wallet before their first stake; surfaces faucet (faucet.vecha.in) and download guidance inline.

6 Authentication & Session System

Pooz supports three identity providers unified behind a single JWT session stored in `localStorage` (`pc_token`, `pc_user`, plus admin token in `sessionStorage.pc_admin_token`).

METHOD	FLOW	ENTRY POINTS
Email	<code>signup/login</code> → JWT → <code>AuthContext.saveSession</code>	POST <code>/auth/signup</code> · <code>/auth/login</code>
Google OAuth	Backend redirect → <code>/auth/google/success?token=...</code> page persists token, fetches <code>/auth/me</code> , reloads to Dashboard	GET <code>/auth/google</code> (server) · <code>/auth/me</code>
VeWorld wallet	GET <code>/auth/nonce/:address</code> → sign Cert(agreement, nonce message) → POST <code>/auth/wallet</code> {address, signature, signer}	GET <code>/auth/nonce/:addr</code> · POST <code>/auth/wallet</code>
Link wallet to existing account	Email/Google session + wallet cert → POST <code>/auth/link-wallet</code> refreshes session with <code>walletUser: true</code> + <code>walletAddress</code>	POST <code>/auth/link-wallet</code>
Password reset	<code>forgot-password</code> (email link) → <code>reset-password</code> (token + strength-validated new password)	POST <code>/auth/forgot-password</code> · <code>/auth/reset-password</code>

6.1 Session Invariants

- `AuthProvider` hydrates the user on mount; a failed `/auth/me` purges stale tokens and renders the app in logged-out state.
- The axios instance attaches `Authorization: Bearer <token>` on every request and, on **401**, clears the session and redirects to `/` (server-side logouts propagate instantly).
- Wallet users are displayed by shortened address (`0x1234...ABCD`); email/Google users by display name.

6.2 Profile & Account Management

Authenticated users can review their profile (rank, streak, stats, linked wallet address), change their password (POST `/auth/change-password`), toggle display identity, and link or replace their wallet from the Wallet / Profile pages.

7 Real-Time Data Pipeline

7.1 Socket.IO Event Bus

On login, AuthContext opens a Socket.IO connection and subscribes the user id so the backend can push personalized and global events:

SOCKET EVENT	BROWSER SIDE-EFFECT	CONSUMER
join_user	emitted with user id on connect	Server routing
notification	<code>window.dispatchEvent(CustomEvent("pc_notification"))</code>	Topbar / recent-activity feeds
market_created	<code>dispatchEvent(new Event("pc_market_update"))</code>	Markets reload hooks
market_resolved	same <code>pc_market_update</code>	Odds/payout views refresh

The `ping / pong` and disconnect handling are managed by the client sdk; the provider tears the socket down on logout.

7.2 Live Price Telemetry

- **VET/USD** — polled from `GET /price/vet` every 5 minutes (initial seed 0.045 USD); fed into `toUsd()` for USD-denominated stake/payout/balance formatting (with K/M auto-scaling).

7.3 Live On-Chain Wallet Balance

- When a user has a linked wallet, the VET balance is read directly from the VeChain node every 30 s (`GET {NODE_URL}/accounts/:address` → `balance / 1e18`).
- The dashboard prefers this live balance over the server-mirrored `vetBalance` whenever available.

8 Admin Panel & Governance

The operational console (`/admin`, `src/pages/admin/AdminPanel.js`) is gated by a secret-code handshake (`POST /auth/admin-verify` → short-lived admin token in session storage). Its five sections:

8.1 Overview

- Stats grid: total users, active markets, total predictions, all-time VET staked, VET in active pools, platform fees (1.5%), weekly volume, resolved markets.
- Top market by pool + on-chain **platform fee withdrawal**: reads the best available fee figure (backend vs `GET /api/admin/contract-stats`), then signs `withdrawFees(adminWallet)` and records the withdrawal.

8.2 Markets

- **Create** — form validates future close time, encodes `createMarket` calldata, signs via Connex, parses the emitted market id from the receipt, then persists DB market with `contractMarketId` + `txHash`.
- **Push / Resync** — retro-fits on-chain registration onto DB-only markets.
- **Resolve** — resolves YES/NO on-chain (`resolveMarket`) when a market has a contract id, then finalizes DB state so payouts are computed ("1.5% platform fee ... winners receive proportional payouts").
- **Cancel** — marks cancelled, making stakes refundable.
- Search + status pills (active / closed / resolved / cancelled).

8.3 Users, Transactions, Notifications

- **Users** — search, promote/demote admins, deactivate/reactivate accounts, inspect auth method, win rate and VET earned.
- **Transactions** — full ledger of deposits, withdrawals, stakes, wins, refunds and fees (statuses confirmed/failed/pending).
- **Notifications** — broadcast typed notifications (system, market_closing, win, deposit/reward) to all active users with a live preview; delivered over the socket plus inbox persistence.

9 Frontend Architecture

9.1 Routing & Page Map

Routing is pathname-based without a router library (`src/App.js`): auth state gates the app, and a fixed set of paths renders dedicated views:

PATH	VIEW	AUTH REQUIRED
/	Landing (logged out) ↔ Dashboard (logged in)	mixed
/admin	Admin Panel	admin code
/forgot-password	Request reset link	—
/reset-password?token=	Set new password	token
/auth/google/success?token=	OAuth token capture → redirect /	—
/market?id=	Share-landing anchor for market spotlight	—
anything else	404 screen ("Back to PredictChain")	—

9.2 Global State & Data Hooks

AuthContext

user, vetPrice, liveBalance, socket;
login/logout/link/refresh; toUsd(); auth-
method-aware identity helpers.

useMarkets

page-able market lists with
loading/error/total + reload.

useMyPredictions

the user's positions, status-filterable.

useWallet

balance + latest 20 transactions
(parallel fetch).

useLeaderboard

rankings by earnings/accuracy;
personal rank strip.

useNotifications

inbox + unread count, mark-all-read.

useProfile

profile edition + persistence.

9.3 Landing Experience

A single-page marketing funnel (`src/pages/Landing.js`) with: animated hero (rotating word swap), live market grid with category filter tabs (falls back to curated static cards when the API is unreachable), "How it works", animated stats cube & leaderboard, VET token/value section, security pillars, live-activity feed, testimonials, FAQ accordion, newsletter capture, and final CTA — all driving toward the unified auth modal.

9.4 Dashboard Experience

The logged-in shell (`src/pages/Dashboard.js`) injects its own design system (CSS custom properties, light/dark themes) and exposes sidebar sections: **Home** (stats, active predictions, quick bet, performance ring, trending), **Markets**, **My Predictions** (with claim actions), **Leaderboard**, **Wallet** (balance, linked address, deposit/withdraw guidance, transactions), **Profile** (badges, stats, password, wallet linking), and **Support** (FAQ + contact cards). A top-bar provides search, notifications dropdown, and theme toggle.

10 Developer Guide & Local Setup

Prerequisites

- **Node.js** ≥ 18 (tested on LTS paths); npm bundled.
- **VeWorld** wallet (desktop extension or mobile app) — download at vieworld.net.
- **Testnet funds** — VET/VTHO from faucet.vecha.in.
- A running **backend** exposing the REST + Socket.IO endpoints (default `localhost:5001`).

Installation & Environment

```
npm install
```

Create `.env` in the project root (CRA convention; `.env` is git-ignored):

```
# REST API base URL (backend)
REACT_APP_API_URL=http://localhost:5001/api
# Socket.IO server for realtime
REACT_APP_SOCKET_URL=http://localhost:5001
# VeChain node for balance + receipt polling
REACT_APP_VECHAIN_NODE=https://node-testnet.vechain.energy
# Prediction smart contract (testnet)
REACT_APP_CONTRACT_ADDRESS=<deployed-contract-address>
# Fallback admin withdrawal target
REACT_APP_ADMIN_WALLET=<admin-address>
```

Run & Build

```
npm start      # development server → http://localhost:3000
npm run build   # production build (react-scripts)
```

Verification Playbook

1. Open `http://localhost:3000` — landing renders with static markets when the API is offline.
2. Sign up (email or Google) and link a VeWorld wallet via the connect prompt.
3. Fund the wallet from the faucet, then place a ≥ 1 VET YES/NO bet and observe the three modal states (Confirm → Wait for confirmation → Predict recorded).
4. From the admin panel, create a market (sign the createMarket tx), Resync/push, then Resolve — validate the receipt topic parsing yields a usable `contractMarketId`.
5. Win path: claim a won prediction and confirm the VET lands in the wallet and the tx history shows a `win` + claim.
6. Trigger a broadcast notification from admin and confirm the live socket push renders in the topbar.

A. REST API surface consumed by the frontend

GROUP	ENDPOINTS
Auth	POST /auth/signup · /auth/login · /auth/wallet · /auth/link-wallet · /auth/forgot-password · /auth/reset-password · /auth/change-password · /auth/admin-verify GET /auth/nonce/:address · /auth/me
Markets	GET /markets · GET /markets/:id · GET /markets/meta/trending
Predictions	GET /predictions/my · POST /predictions · POST /predictions/:id/claim
Wallet	GET /wallet/balance · GET /wallet/transactions · POST /wallet/deposit · POST /wallet/withdraw
Leaderboard	GET /leaderboard
User	GET /users/profile · PATCH /users/profile · GET /users/notifications · PATCH /users/notifications/read-all
Price	GET /price/vet
Admin	GET /admin/stats · /admin/markets · /admin/users · /admin/transactions POST /admin/markets · /admin/markets/:id/resolve · /admin/markets/:id/cancel · /admin/withdraw-fees · /admin/notify-all PATCH /admin/markets/:id · /admin/users/:id/toggle-admin · /admin/users/:id/toggle-active
Ops	GET /api/admin/contract-stats (contract fee reconciliation)

B. On-chain call summary

SELECTOR	SIGNATURE	VALUE SENT	SOURCE FILE
0xebd389fe	predict(uint256,bool)	stake (wei)	vechain.js
0x677bd9ff	claim(uint256)	0	vechain.js
0x5b7baf64	refund(uint256)	0	vechain.js
0x164e68de	withdrawFees(address)	0	vechain.js / AdminPanel
0xc48d8656	createMarket(string,string,uint256,bool)	0	AdminPanel
0x57bde446	resolveMarket(uint256,bool)	0	AdminPanel

C. Repository map

```
vetpredict/ (repo: predictchain-frontend)
├─ package.json / package-lock.json
├─ public/index.html
├─ src
│   ├── index.js           React bootstrap (AuthProvider → App)
│   ├── App.js             pathname routing + auth gate
│   ├── context/AuthContext.js session, socket, price, toUsd, wallet linking
│   ├── hooks/useData.js   useMarkets · useMyPredictions · useWallet · useLeaderboard ·
│   └─ useNotifications · useProfile
│       ├── services/api.js   axios API client (JWT, 401 purge)
│       ├── services/vechain.js Connex/VeWorld signer, selectors, receipt polling
│       └─ pages
│           ├── Landing.js     marketing funnel + auth modal
│           └─ Dashboard.js    app shell
│   (Home/Markets/Predictions/Leaderboard/Wallet/Profile/Support)
│       ├── GoogleAuthSuccess.js OAuth token capture
│       ├── PasswordReset.js    forgot/reset flows
│       └─ admin/AdminPanel.js  secret-gated ops console
```

D. Known divergences & follow-ups

- FAQ copy says "2% platform fee"; implementation is 1.5% — reconcile copy.
- Two node endpoints are in play (energy + org); consolidate into a fallback chain for resilience.
- Selected contract signatures are hard-pinned in the client; consider a shared ABI/types package to prevent drift.
- CRA (react-scripts) is near end-of-life; a migration path to Vite or Next is worth planning.